Control plane and etcd internals

How a request travels through the API server, how etcd stores and protects state, and how the scheduler, controllers and kubelet turn it into running pods.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. The request path: authentication, authorization and admission (12 min)
2. API machinery: versions, watches and server-side apply (12 min)
3. etcd: data model, Raft and quorum (12 min)
4. etcd operations: health, compaction and defragmentation (12 min)
5. The scheduler: filtering, scoring and binding (12 min)
6. Controllers: the reconcile loop and leader election (12 min)
7. The kubelet: pod lifecycle and the container runtime interface (12 min)
8. Control plane failure modes and diagnosis (13 min)
Cheat sheet

Lesson 1. The request path: authentication, authorization and admission

12 minutes. Follow one API request through authentication, authorization, mutating and validating admission and storage, and learn where each stage can reject or slow it.

Every change to a Kubernetes cluster is an HTTPS request to the API server. The server does not simply write what it receives. It passes each request through a fixed series of stages, and knowing the order explains most permission errors, webhook outages and surprising defaults.

What you will learn

- The order of stages: authentication, authorization, mutating admission, schema validation, validating admission, storage
- How each stage can say no, and what the error looks like
- Why admission webhooks are powerful and risky
- How to read the stage from an error message

Authentication: who is asking

The API server first works out the identity. It tries each configured authenticator until one succeeds: client certificates, bearer tokens (service account tokens), OpenID Connect tokens, or a webhook. If none succeeds, the request is rejected with 401 Unauthorized. A successful authenticator produces a user name, groups and extra attributes. Nothing about permissions is decided yet. A request with no credentials can still become the user `system:anonymous` if anonymous access is enabled, which is why hardening guides turn it off.

Authorization: may they do this

Next the server asks the authorizers whether this identity may perform this verb on this resource in this namespace. A kubeadm cluster uses Node authorization and RBAC. The first authorizer that allows the request wins, and if none allows it the answer is 403 Forbidden. The message names the user, the verb, the resource and the namespace, which is usually enough to write the missing rule. You can test any identity with `kubectl auth can-i`.

Admission: may the object be changed or accepted

Only an authorized request reaches admission. Mutating admission plugins and webhooks run first and may change the object, for example by adding a sidecar or a default value. The server then validates the object against its schema. Validating admission runs last and can only accept or reject. Built-in plugins include LimitRanger, ResourceQuota, ServiceAccount and Pod Security. Admission webhooks let your own service make the decision, and ValidatingAdmissionPolicy lets you write rules in CEL without running a service.

Storage and the response

If every stage passes, the API server writes the object to etcd and returns the stored version with its new resourceVersion. Controllers then see the change through watches. Reads skip the admission stages, but still go through authentication and authorization.

Reading errors by stage

A 401 is authentication, a 403 is authorization, and a message that starts with "admission webhook ... denied the request" or "admission plugin ... denied" is admission. A 422 usually means the object failed schema validation. A timeout on every create is a classic sign of a webhook that is slow or unreachable and configured with failurePolicy Fail.

```
kubectl auth can-i create pods -n dev --as jane
kubectl get validatingwebhookconfigurations
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
kubectl get mutatingwebhookconfigurations
```

Diagram: The request path: authentication, authorization and admission

Part	What it does	What goes wrong
Client	Sends the request.	Wrong credentials.
Authentication	Identifies the user.	Anonymous allowed.
Authorization	RBAC and Node rules.	Too broad roles.
Mutating admission	May change the object.	Webhook slow or down.
Validating admission	Accepts or rejects.	Policy blocks valid change.
etcd	Stores the object.	No quorum.

Flow: Client to Authentication / Authentication to Authorization / Authorization to Mutating admission / Mutating admission to Validating admission / Validating admission to etcd

Incident: The webhook that blocked every pod

Incident: *A security webhook went down and nobody could create pods anywhere in the cluster.*

```
$ kubectl run test --image=nginx
Error from server (InternalError): Internal error occurred: failed calling webhook "policy.example \
    .com": Post "https://policy.svc:443/validate": dial tcp 10.0.4.12:443: connect: connection ref \
    used
$ kubectl get validatingwebhookconfigurations policy -o jsonpath='{.webhooks[0].failurePolicy}'
Fail
(this is a simulated illustration, not a real incident)
```

The webhook had failurePolicy Fail, so while its pods were down the API server refused every matching request, including the pods needed to bring the webhook back. The recovery was to delete or edit the webhook configuration so it ignored failures. Scope webhooks with namespace selectors, exclude the namespace that runs the webhook itself, and prefer failurePolicy Ignore for non-critical checks.

Common mistakes

- Reading a 403 as an admission problem, or the reverse
- Giving a webhook failurePolicy Fail without excluding its own namespace
- Assuming reads go through admission
- Forgetting that mutating changes happen before validation
- Granting broad RBAC to fix a webhook error

Check yourself

Which stage runs immediately after authentication?

Answer: Authorization

A request returns 403 Forbidden naming a user, verb and resource. What stage rejected it?

Answer: Authorization

What runs first, mutating or validating admission?

Answer: Mutating admission

Why is failurePolicy Fail risky for a webhook?

Answer: If the webhook is unreachable, matching requests are rejected

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 2. API machinery: versions, watches and server-side apply

12 minutes. Understand API groups and versions, resourceVersion, watches, optimistic concurrency and server-side apply so controllers and tools behave predictably.

The API server is a typed, versioned store with a change feed. Controllers, kubectl and operators all rely on the same few ideas: resources have versions, every object has a resourceVersion, and clients can watch for changes instead of polling. This lesson explains those ideas and why they matter for cluster stability.

What you will learn

- API groups, versions and how conversion keeps old clients working
- resourceVersion, list and watch, and why watches scale
- Optimistic concurrency and 409 conflicts
- Server-side apply and field ownership
- API Priority and Fairness protecting the server

Groups, versions and deprecation

Resources live in API groups such as `apps/v1` or `networking.k8s.io/v1`. A resource can be served in several versions at once, and the server converts between them. Stored objects use one storage version. Kubernetes deprecates versions on a published schedule, and `kubectl api-resources` and `kubectl api-versions` show what your cluster serves. Before an upgrade, check manifests and controllers for removed versions, because a removed version fails at apply time.

resourceVersion, list and watch

Each object carries a resourceVersion that comes from etcd revisions. A client lists a resource type once, remembers the resourceVersion of the list, then opens a watch from that point. The server streams add, modify and delete events, so thousands of controllers can follow changes without polling. If a client falls too far behind, the watch returns an error saying the version is too old, and the client must list again. Informers in client libraries do this for you and keep a local cache.

Optimistic concurrency

Updates include the resourceVersion the client last saw. If someone changed the object since, the server answers 409 Conflict and the client must read and retry. This is why two tools editing the same object sometimes collide. It is also why controllers re-read before writing.

Server-side apply

With server-side apply the server tracks which manager owns each field. `kubectl apply --server-side` sends your intended fields, and the server merges them and reports a conflict if another manager owns a field you are changing. This makes it safe for several controllers to manage different parts of one object, and it is the preferred way for new tooling.

```
kubectl api-resources --api-group=apps
kubectl get deploy web -o yaml --show-managed-fields | head -40
kubectl apply --server-side -f deploy.yaml
```

API Priority and Fairness

The server limits concurrent requests by sorting them into flows with priority levels, so one noisy client cannot starve system controllers. Clients that receive 429 Too Many Requests should back off. Metrics such as apiserver_flowcontrol_rejected_requests_total show when limits are hit.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: API machinery: versions, watches and server-side apply

Part	What it does	What goes wrong
Controller	Lists then watches.	Polling in a loop.
API server	Serves and converts versions.	Overloaded.
Priority and fairness	Queues by flow.	Misconfigured levels.
Watch cache	Streams changes.	Client too far behind.
etcd	Source of revisions.	Slow disk.

Flow: Controller to API server / API server to Priority and fairness / Priority and fairness to Watch cache / Watch cache to etcd

Incident: A controller that polled instead of watched

Incident: *A custom tool listed every pod once a second and slowed the whole API.*

```
$ kubectl get --raw /metrics | grep apiserver_flowcontrol_rejected_requests_total | head -2
apiserver_flowcontrol_rejected_requests_total{flow_schema="catch-all",priority_level="catch-all",r \
    eason="queue-full"} 412
$ kubectl get pods --all-namespaces --no-headers | wc -l
2840
(this is a simulated illustration, not a real incident)
```

The tool used a full list every second from a service account that fell into the catch-all flow, so its requests were queued and then rejected, and the extra load made other clients slower. Rewriting it with an informer, which lists once and then watches, cut the request rate by more than ninety percent. Always watch rather than poll, and give important controllers their own flow schema.

Common mistakes

- Polling the API with repeated list calls
- Ignoring 409 conflicts instead of re-reading
- Applying manifests that use removed API versions
- Mixing client-side and server-side apply on the same fields
- Not backing off after a 429

Check yourself

What does a client do to follow changes efficiently?

Answer: List once, then watch from the returned resourceVersion

What does HTTP 409 Conflict on an update mean?

Answer: The object changed since the client read it

What does server-side apply add?

Answer: Field ownership tracked by the server

What should you check before an upgrade?

Answer: That manifests and controllers do not use removed API versions

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. etcd: data model, Raft and quorum

12 minutes. See how Kubernetes stores objects in etcd, how Raft elects a leader and commits writes, and how to size and reason about quorum.

etcd is the only stateful part of the control plane. If it is healthy the API server can be replaced freely. If it is lost without a backup, the cluster state is gone. Understanding how it stores data and agrees on writes makes sizing and recovery decisions straightforward.

What you will learn

- How objects are stored as keys under /registry
- Revisions and MVCC
- Leader election and commit in Raft
- Quorum maths and why odd member counts are used
- What disk and network latency do to the cluster

Keys and revisions

etcd is a key-value store. The API server writes each object under a key such as `/registry/pods/default/web-1`, as a protobuf encoded value. Every write creates a new revision, a cluster-wide counter, and etcd keeps old revisions until they are compacted. This multi-version design is what makes watches possible, because a client can ask for all changes since a given revision. The resourceVersion you see in objects comes from these revisions.

Raft in plain words

Members of an etcd cluster elect one leader using the Raft algorithm. All writes go to the leader, which appends them to its write-ahead log and replicates them to followers. A write is committed when a majority of members have stored it. If the leader fails, the followers hold a new election after a timeout. Reads can be served linearizably through the leader, which is what the API server asks for by default.

Quorum

A majority is floor(n/2) + 1. Three members tolerate one failure, five tolerate two, and seven tolerate three. An even number adds cost without adding tolerance: four members still tolerate one failure. When quorum is lost the cluster cannot commit writes, so the API server cannot change anything, although running pods keep running.

Latency matters

Every commit waits for a disk sync on a majority of members, so slow disks directly slow the whole cluster. Use fast local SSD storage for etcd, keep members close in network terms, and avoid sharing the disk with noisy workloads. Slow fsync shows up as wal_fsync_duration warnings in the logs and as leader changes.

```
def tolerated(members):
    quorum = members // 2 + 1
    return members - quorum

for n in (1, 2, 3, 4, 5, 7):
    print(n, "members tolerate", tolerated(n), "failure(s)")
```

Look at it

```
ETCDCTL_API=3 etcdctl endpoint status -w table --endpoints=https://127.0.0.1:2379 --cacert=/etc/ku \
    bernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/ \
    etcd/server.key
```

The table shows the leader, the database size and the raft term for each member.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: etcd: data model, Raft and quorum

Part	What it does	What goes wrong
API server	Writes objects.	Cannot reach etcd.
Leader	Orders and replicates writes.	Slow disk causes elections.
Follower	Stores the log.	Lagging member.
Follower	Stores the log.	Lost member.
WAL on disk	Synced before commit.	High fsync latency.

Flow: API server to Leader / Leader to Follower / Leader to Follower / Leader to WAL on disk

Incident: Two of three members down

Incident: *Two etcd members failed in a single maintenance window and the API became read only in practice.*

```
$ etcdctl endpoint health --endpoints=https://10.0.0.1:2379,https://10.0.0.2:2379,https://10.0.0.3 \
    :2379
https://10.0.0.1:2379 is healthy: successfully committed proposal: took = 8.2ms
https://10.0.0.2:2379 is unhealthy: failed to commit proposal: context deadline exceeded
https://10.0.0.3:2379 is unhealthy: failed to commit proposal: context deadline exceeded
(this is a simulated illustration, not a real incident)
```

With one of three members left there was no quorum, so no writes could be committed and kubectl changes hung. Running pods were unaffected. The recovery was to bring at least one member back with its data, restoring quorum, then replace the other. If both members had lost their data, a snapshot restore would have been needed. Do maintenance one member at a time and check health between steps.

Common mistakes

- Running an even number of members
- Putting etcd on slow or shared disks
- Taking down two members at once for maintenance
- Stretching members across high-latency links
- Treating etcd as a database for application data

Check yourself

How many failures does a five member etcd cluster tolerate?

Answer: Two

When is a write committed?

Answer: When a majority of members have stored it

What happens to running pods when etcd loses quorum?

Answer: They keep running, but no cluster changes can be committed

Why is disk latency important for etcd?

Answer: Each commit waits for a disk sync on a majority of members

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. etcd operations: health, compaction and defragmentation

12 minutes. Keep etcd healthy: check status and alarms, understand compaction and defragmentation, and recover from a space quota alarm.

Most etcd incidents are boring and preventable: a database that grew past its quota, a slow disk, or a member that fell behind. This lesson covers the routine checks and the exact steps to recover from the most common failure, the space quota alarm.

What you will learn

- Reading endpoint status, health and alarms
- Why the database grows and what compaction does
- Defragmentation and when it is safe
- The space quota alarm and how to clear it
- Metrics worth alerting on

Routine checks

Run three commands regularly, with the same endpoint and certificate flags you use for backups. `endpoint health` tests that each member can commit. `endpoint status` shows database size, leader and raft index for each member. `alarm list` shows any active alarm.

```
export ETCDCTL_API=3
E="--endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kuberne \
    tes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key"
etcdctl $E endpoint health
etcdctl $E endpoint status -w table
etcdctl $E alarm list
```

Why the database grows

Because etcd keeps old revisions, the data file grows with every change, and objects that change often, such as leases and events, create many revisions. Compaction discards revisions older than a chosen one. The API server compacts automatically on a schedule, by default every five minutes, so growth normally stays bounded.

Compaction is not defragmentation

Compaction marks old revisions as free, but the file on disk does not shrink. Free space inside the file is reused for new data. Defragmentation rewrites the file to give space back to the filesystem. It blocks the member while it runs, so defragment one member at a time, and not the leader first.

```
etcdctl $E defrag --endpoints=https://127.0.0.1:2379
```

The space quota alarm

etcd has a backend quota, 2 GiB by default and commonly raised to 8 GiB. If the data file reaches it, etcd raises a NOSPACE alarm and rejects writes, with the error "mvcc: database space exceeded". Recovery is: find the current revision, compact to it, defragment each member, then disarm the alarm.

```
REV=$(etcdctl $E endpoint status -w json | grep -o '"revision":[0-9]*' | head -1 | cut -d: -f2)
etcdctl $E compact $REV
etcdctl $E defrag
etcdctl $E alarm disarm
```

What to alert on

Alert on database size close to the quota, on a rising rate of leader changes, on high wal_fsync and backend_commit latency, and on failed proposals. Also watch the number of objects, since a runaway controller that creates many events or secrets is the usual cause of growth.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: etcd operations: health, compaction and defragmentation

Part	What it does	What goes wrong
Health and status	Weekly checks.	Skipped.
Database growth	Revisions pile up.	Hot objects.
Compaction	Frees old revisions.	Not run.
Defragmentation	Returns disk space.	Run on all at once.
NOSPACE alarm	Rejects writes.	Not disarmed.

Flow: Health and status to Database growth / Database growth to Compaction / Compaction to Defragmentation / Defragmentation to NOSPACE alarm

Incident: Database space exceeded

Incident: *Cluster changes started failing with an etcd space error after a noisy controller created millions of events.*

```
$ kubectl create namespace test
Error from server: etcdserver: mvcc: database space exceeded
$ etcdctl alarm list
memberID:8e9e05c52164694d alarm:NOSPACE
(this is a simulated illustration, not a real incident)
```

The data file had reached its quota and etcd had gone into protection mode. The team compacted to the current revision, defragmented members one at a time, disarmed the alarm and then fixed the controller that created the events. Raising the quota alone would only have delayed the next alarm. Always find and fix the writer.

Common mistakes

- Defragmenting every member at once
- Raising the quota without finding the cause
- Forgetting to disarm the alarm after cleaning up
- Not checking health after maintenance
- Ignoring growth until writes fail

Check yourself

What does compaction do?

Answer: Discards old revisions so their space can be reused

What does defragmentation do?

Answer: Rewrites the data file to return free space to the filesystem

What is the sequence to recover from the NOSPACE alarm?

Answer: Compact, defragment, then disarm the alarm

Why find the cause of growth?

Answer: Otherwise the database will fill again

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. The scheduler: filtering, scoring and binding

12 minutes. See how the scheduler picks a node: the queue, filter and score plugins, preemption and binding, and how to read why a pod stays Pending.

The scheduler has one job: assign each unscheduled pod to a node. It does this in a predictable cycle, and every decision leaves evidence in pod events. Once you know the stages, a Pending pod stops being mysterious.

What you will learn

- The scheduling queue and the cycle of filter, score and bind
- What the main filter and score plugins check
- Preemption and priority
- Scheduler profiles and extension points
- Reading FailedScheduling events

The cycle

Pods without a node name enter a queue ordered by priority. For each pod the scheduler runs a scheduling cycle. First, filtering removes nodes that cannot run the pod. Second, scoring ranks the remaining nodes. The best node is chosen, with ties broken at random. Finally, a binding cycle writes the node name into the pod through the API server, and the kubelet on that node takes over.

Filtering

Filter plugins check hard constraints. NodeResourcesFit checks that requests fit the free capacity, NodeAffinity and NodeSelector check labels, TaintToleration checks taints, VolumeBinding checks that claims can be satisfied and PodTopologySpread checks skew limits. A node that fails any filter is out. If every node is filtered out the pod stays Pending and the scheduler records an event that counts the reasons.

Scoring

Score plugins rank nodes by preference, for example spreading pods across nodes, favouring nodes that already have the image, or balancing resource use. Scores are weighted and summed. Scoring cannot rescue a pod that failed filtering, it only chooses between valid nodes.

Preemption

If no node fits and the pod has a higher priority than some running pods, the scheduler can choose a node where evicting lower priority pods would make room. It marks the victims for deletion and records a nominated node for the new pod. Priority classes control this, and critical system pods use very high priorities.

Profiles and extension

The scheduling framework exposes extension points such as filter, score and bind. A scheduler configuration file can enable or disable plugins and set weights, and several profiles can run in one scheduler, selected by `schedulerName` in the pod. Running a second scheduler is possible but rarely needed.

```
kubectl describe pod web-1 | sed -n '/Events/,$p'
kubectl get pods -n kube-system -l component=kube-scheduler
kubectl logs -n kube-system -l component=kube-scheduler --tail=20
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: The scheduler: filtering, scoring and binding

Part	What it does	What goes wrong
Queue	Ordered by priority.	Starved low priority.
Filter	Removes unfit nodes.	Taints and selectors.
Score	Ranks the rest.	Skewed weights.
Bind	Writes the node name.	API conflict.
kubelet	Starts the pod.	Image pull errors.

Flow: Queue to Filter / Filter to Score / Score to Bind / Bind to kubelet

Incident: A scheduler that stopped scheduling

Incident: *A typo in a scheduler configuration file left every new pod Pending while existing pods ran normally.*

```
$ kubectl get pods -n kube-system -l component=kube-scheduler
NAME                              READY   STATUS             RESTARTS   AGE
kube-scheduler-cp1                0/1     CrashLoopBackOff   14         32m
$ kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
web-1   0/1     Pending   0          9m
(this is a simulated illustration, not a real incident)
```

The scheduler is a static pod, and an edit to its configuration file had an unknown field, so the process exited at start. Pods without a node stayed Pending because nothing was binding them, with no scheduling events at all, which is the clue. Restoring the backed-up manifest brought it back. An empty events list on a Pending pod means the scheduler did not see it, not that it rejected it.

Common mistakes

- Changing resource limits when the event names a taint
- Assuming scoring can override a failed filter
- Editing a scheduler manifest without keeping a backup
- Reading Pending as a node problem when the scheduler is down
- Forgetting that priority only matters when capacity is short

Check yourself

What does filtering do?

Answer: Removes nodes that cannot run the pod

A Pending pod has no events at all. What is likely?

Answer: The scheduler is not running or not seeing the pod

What decides between several valid nodes?

Answer: Scoring

When can preemption happen?

Answer: When a higher priority pod cannot fit and lower priority pods could be evicted

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 6. Controllers: the reconcile loop and leader election

12 minutes. Learn how controllers drive the cluster toward the desired state with informers, work queues and reconcile loops, and how leader election keeps one active copy.

Kubernetes is a set of control loops. You declare what you want, and controllers keep comparing it with what exists and acting on the difference. When something does not scale, restart or clean up, the explanation is almost always in a controller, so it is worth knowing how they work.

What you will learn

- The observe, compare, act loop and level-triggered design
- Informers, caches and work queues
- The built-in controllers and the controller manager
- Owner references and garbage collection
- Leader election with Leases

Level-triggered control

A controller does not react to each event as a command. It reads the current desired state and the current actual state and acts to close the gap. If it misses an event or restarts, it simply looks again, so the system recovers by design. This is why Kubernetes objects have a spec for what you want and a status for what exists.

Informers and work queues

A controller uses an informer to list and watch a resource type and keep a local cache. Changes put the object key into a rate limited work queue. A worker takes a key, reads the object from the cache, reconciles, and either finishes or requeues with backoff. Using the cache keeps load off the API server.

The controller manager

The kube-controller-manager runs many controllers in one process: the Deployment, ReplicaSet, Job, Node, Endpoints and ServiceAccount controllers, among others. A Deployment controller creates ReplicaSets, a ReplicaSet controller creates pods, and so on, so a single scale request fans out through several loops. The cloud-controller-manager does the same for cloud specific resources such as load balancers and node lifecycle.

Owner references and garbage collection

Objects created by a controller carry an ownerReference to their parent. When the parent is deleted, the garbage collector deletes dependents, in background by default or foreground if requested. That is why deleting a Deployment removes its pods, and why an orphaned object may point to an owner that no longer exists.

Leader election

Control plane components usually run more than one copy for availability, but only one controller manager or scheduler may act at a time. Each copy tries to hold a Lease object in kube-system, renewing it before it expires. The holder is the leader. If it stops renewing, another copy takes over after the lease duration.

```
kubectl get lease -n kube-system kube-controller-manager -o jsonpath='{.spec.holderIdentity}{"\n"} \
    '
kubectl get lease -n kube-system
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Controllers: the reconcile loop and leader election

Part	What it does	What goes wrong
Desired state	Written by users.	Wrong spec.
Informer cache	Watches and caches.	Stale cache.
Work queue	Rate limited keys.	Hot loop.
Reconcile	Closes the gap.	Error loops.
Act via API	Creates or deletes objects.	Permission denied.

Flow: Desired state to Informer cache / Informer cache to Work queue / Work queue to Reconcile / Reconcile to Act via API

Incident: Leader flapping on a loaded control plane

Incident: *The controller manager kept changing leaders, and deployments scaled slowly.*

```
$ kubectl get lease -n kube-system kube-controller-manager -o jsonpath='{.spec.leaseTransitions}'
187
$ kubectl logs -n kube-system -l component=kube-controller-manager --tail=3
E1114 08:12:01 leaderelection.go:332] error retrieving resource lock kube-system/kube-controller-m \
    anager: context deadline exceeded
(this is a simulated illustration, not a real incident)
```

The active controller could not renew its Lease because API requests to the server were slow, so it lost leadership, and another copy had to start its caches before acting. Each transition delayed reconciliation. The cause was an overloaded API server with a slow etcd disk. Fixing the storage latency stopped the flapping. Leader election problems usually point to slow API or etcd rather than a bug in the controller.

Common mistakes

- Expecting controllers to act on a single event rather than on state
- Editing a ReplicaSet owned by a Deployment and expecting it to stick
- Ignoring leaseTransitions as a health signal
- Deleting an owner without checking the dependents
- Running controllers that write every second without backoff

Check yourself

What does a level-triggered controller do after a restart?

Answer: Reads current and desired state again and acts on the difference

What creates the pods for a Deployment?

Answer: The ReplicaSet controller, through a ReplicaSet the Deployment controller made

How is leader election implemented for control plane components?

Answer: By holding and renewing a Lease object

What happens to dependents when an owner is deleted?

Answer: The garbage collector deletes them

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. The kubelet: pod lifecycle and the container runtime interface

12 minutes. Understand what the kubelet does on each node: its sync loop, pod phases and probes, the CRI, CNI and CSI calls, node heartbeats and static pods.

The control plane decides, and the kubelet does. It is the agent on every node that turns pod specifications into running containers and reports what is actually happening. Most node level problems, from pods stuck in ContainerCreating to NotReady nodes, are kubelet problems.

What you will learn

- The sync loop and how the kubelet gets its pods
- The CRI, CNI and CSI calls during pod start
- Pod phases, container states and probes
- Node status, heartbeats and eviction
- Static pods and the kubelet configuration

Where pods come from

The kubelet watches the API server for pods bound to its node. It also reads static pod manifests from a local directory and optionally an HTTP source. For each pod it runs a sync loop that compares the desired containers with what the runtime reports and takes action to match.

Starting a pod

The kubelet asks the container runtime, through the Container Runtime Interface, to create a pod sandbox. A CNI plugin then sets up the pod network and assigns an IP. Volumes are mounted, with CSI drivers for dynamic storage. Images are pulled, then containers are created and started in order, init containers first. Each step can fail visibly: ContainerCreating with a network or mount event, ImagePullBackOff, or CrashLoopBackOff.

Phases, states and probes

A pod phase is Pending, Running, Succeeded, Failed or Unknown. Each container is Waiting, Running or Terminated, with a reason. Startup, liveness and readiness probes let the kubelet decide whether to restart a container or whether to include it in service endpoints. A failing readiness probe removes the pod from endpoints without restarting it.

Node status and heartbeats

The kubelet updates the node object conditions such as Ready, MemoryPressure and DiskPressure, and renews a Lease in the kube-node-lease namespace as a cheap heartbeat. If the control plane stops seeing heartbeats for the grace period, the node controller marks the node NotReady and later taints it so pods are evicted. Under pressure the kubelet itself evicts pods, lowest priority and highest overuse first.

Configuration and diagnosis

The kubelet runs as a systemd service on the host, not as a pod. Its settings live in a config file, typically `/var/lib/kubelet/config.yaml`.

```
systemctl status kubelet
journalctl -u kubelet --since "10 min ago" | tail -20
crictl ps -a | head
grep staticPodPath /var/lib/kubelet/config.yaml
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: The kubelet: pod lifecycle and the container runtime interface

Part	What it does	What goes wrong
API server	Source of pods.	Unreachable.
kubelet	Sync loop.	Stopped service.
Container runtime	Creates containers.	Runtime down.
CNI plugin	Pod network.	Missing config.
CSI driver	Volumes.	Mount error.

Flow: API server to kubelet / kubelet to Container runtime / kubelet to CNI plugin / kubelet to CSI driver

Incident: The node that went NotReady after a reboot

Incident: *A worker came back from a reboot as NotReady and its pods were rescheduled.*

```
$ kubectl get nodes
NAME       STATUS     ROLES    AGE   VERSION
worker-2   NotReady   <none>   90d   v1.31.2
$ systemctl is-active containerd kubelet
active
inactive
(this is a simulated illustration, not a real incident)
```

The container runtime started, but the kubelet service was not enabled to start at boot, so no heartbeats reached the control plane. After the grace period the node controller marked the node NotReady and evicted its pods. Starting and enabling the kubelet fixed it. Check the lowest layer first: is the service running, and is it enabled.

Common mistakes

- Looking in kubectl before checking the kubelet service on a NotReady node
- Forgetting that readiness failures remove endpoints but do not restart
- Confusing the pod phase with the container state
- Editing static pod manifests without a backup
- Not enabling the kubelet and runtime services at boot

Check yourself

What does the kubelet use to ask for containers to be created?

Answer: The Container Runtime Interface

What happens when a readiness probe fails?

Answer: The pod is removed from service endpoints

How does the control plane know a node is alive?

Answer: Through kubelet heartbeats such as node Lease renewals

Where does the kubelet run on a kubeadm node?

Answer: As a systemd service on the host

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Control plane failure modes and diagnosis

13 minutes. Work out what still works and what stops when each control plane component fails, and follow a fixed order to diagnose a sick control plane.

A control plane failure feels dramatic, but each component has a narrow job, so each failure has a narrow signature. Knowing what keeps working is the fastest route to the cause, and it also tells you how urgent the problem is.

What you will learn

- What breaks and what survives for each component
- Symptoms that point to the API server, etcd, scheduler, controller manager or kubelet
- A fixed diagnosis order
- The tools you can use when kubectl is unavailable

The failure matrix

Running pods keep running in nearly every control plane failure, because the kubelet and the container runtime work from what they already have. What stops is change. If etcd loses quorum, nothing can be written. If the API server is down, nobody can read or write and controllers cannot act, but pods and Services keep working. If the scheduler is down, existing pods run and new pods stay Pending with no events. If the controller manager is down, deployments do not scale, replacements are not created and nodes are never marked NotReady. If a kubelet is down, only that node is affected.

Reading the symptoms

kubectl timeouts or connection refused point to the API server or the network path to it. A "server is currently unable to handle the request" or etcd errors point to etcd. Pending pods without events point to the scheduler. A Deployment that does not create ReplicaSets points to the controller manager. A single NotReady node points to its kubelet, runtime or network.

Diagnosis order

Work from the bottom up. First the node: is the kubelet running, and is the container runtime running. Next the static pods: do the manifests exist, and does `crictl ps` show the containers. Then each component log, starting with etcd, then the API server, then the scheduler and controller manager. Make one change at a time, and re-check the symptom after each.

When kubectl does not work

Use the tools on the control plane node itself. `crictl ps -a` lists containers, `crictl logs` reads their logs, `journalctl -u kubelet` shows why the kubelet could not start a static pod, and `ls /etc/kubernetes/manifests` shows the manifests. Health endpoints can be queried directly.

```
crictl ps -a | grep -E "etcd|apiserver|scheduler|controller"
curl -k https://127.0.0.1:6443/livez?verbose | tail -5
ls -l /etc/kubernetes/manifests
```

A quick reference in code

```
failure = {
    "etcd quorum lost": "no writes, pods keep running",
    "api server down": "no kubectl, pods and services keep running",
    "scheduler down": "new pods Pending with no events",
    "controller manager down": "no scaling, no replacement, no node eviction",
    "one kubelet down": "that node NotReady, others fine",
}
for k, v in failure.items():
    print(k, "->", v)
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Control plane failure modes and diagnosis

Part	What it does	What goes wrong
Symptom	What the user sees.	Vague report.
Node layer	kubelet and runtime.	Service disabled.
Static pods	Manifests and containers.	Bad edit.
Component logs	etcd then API then rest.	Wrong order.
One change	Then re-check.	Many changes at once.

Flow: Symptom to Node layer / Node layer to Static pods / Static pods to Component logs / Component logs to One change

Incident: Deployments that would not scale

Incident: *A team saw deployments ignore scale requests while all existing pods stayed healthy.*

```
$ kubectl scale deploy web --replicas=5
deployment.apps/web scaled
$ kubectl get deploy web
NAME   READY   UP-TO-DATE   AVAILABLE
web    2/2     2            2
$ kubectl get pods -n kube-system -l component=kube-controller-manager
NAME                          READY   STATUS             RESTARTS
kube-controller-manager-cp1   0/1     CrashLoopBackOff   9
(this is a simulated illustration, not a real incident)
```

The API accepted the new replica count, but the controller manager that would create the ReplicaSet change was crash looping because of an expired client certificate path in its manifest. Existing pods ran normally. The log showed the exact file it could not read, and correcting the manifest restored scaling. A request that the API accepts but nothing acts on points to a controller, not the API.

Common mistakes

- Assuming an outage means pods have stopped
- Changing several things before re-checking
- Looking only at kubectl when it may be the failing component
- Skipping the node layer
- Not keeping copies of manifests before edits

Check yourself

If the controller manager is down, what happens?

Answer: Existing pods run, but scaling and replacement do not happen

A new pod is Pending with no events. What is the likely cause?

Answer: The scheduler is down

Which order should you diagnose in?

Answer: From the node layer upward

How do you inspect control plane containers when kubectl fails?

Answer: With crictl and the kubelet journal on the node

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Control plane and etcd

The request path, etcd health commands, component checks and what to do when kubectl is down.

Request path

Stage	Symptom of failure
Authentication	401 Unauthorized
Authorization	403 Forbidden naming the user and verb
Mutating and validating admission	Error naming the plugin or webhook
Schema validation	Invalid value error naming the field

etcd health

Job	Command
Member health	`etcdctl endpoint health --cluster`
Size and leader	`etcdctl endpoint status -w table`
Alarms	`etcdctl alarm list`
Compact	`etcdctl compact $REV`
Defragment one member	`etcdctl defrag --endpoints=$ONE`
Clear alarm	`etcdctl alarm disarm`

Component check

Symptom	Look at
Pending, no events	kube-scheduler and its lease
No scaling, no replacement pods	kube-controller-manager
Writes fail or are slow	etcd disk latency and size
Node NotReady	kubelet, runtime, node lease

When kubectl does not work

Job	Command
Containers on the node	`crictl ps -a`
Kubelet log	`journalctl -u kubelet --since "10 min ago"`
Static pod manifests	`ls /etc/kubernetes/manifests`

MARGA by EknathaLabs
marga.eknathalabs.com